Neural Computing and Applications (2023) 35:22007–22026
https://doi.org/10.1007/s00521-023-08949-4

ORIGINAL ARTICLE

Two-view LSTM variational auto-encoder for fault detection and diagnosis in multivariable manufacturing processes

Li Qi[1] · Yuwei Ren[1] · Yixian Fang[2] · Jinglin Zhou[3]

Received: 29 September 2022 / Accepted: 1 August 2023 / Published online: 25 August 2023
© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2023

Abstract
Process monitoring of industrial production has always been one of the main concerns of process industry systems. As artificial intelligence booms, fault detection and diagnosis via deep learning has been widely used in industrial process monitoring, such as chemical process monitoring flow. Whereas, the current deep framework is difficult to simultaneously reconcile the relationship between different variables and time series caused by the increasing complexity of industrial systems, which reduces the accuracy of fault detection and diagnosis. Considering the significant time series processing potential of long short-term memory (LSTM) networks, this paper proposes a novel two-view LSTM variational auto-encoder, aliased as TVAE. First, the running data in period T is obtained through sliding window, then it utilizes two-view embedding to compress time series information and capture the correlation between time variables, after that it is judged whether it is fault data according to the anomaly score of the variational auto-encoder. Finally, since the encoding process makes the time-dependent and variable dependent features more obvious, the fault characteristic of the encoded data can be obtained by neuron grouping convolution, and then classified for diagnosis. Experimental results on Tennessee Eastman process benchmark show that TVAE is superior to the state-of-the-art methods and provides a new paradigm for industrial process fault detection and diagnosis.

Keywords Fault detection and diagnosis · LSTM auto-encoder · Process monitoring

1 Introduction

Safety and reliability of industrial processes are critical to production and development. Fault detection and diagnosis (FDD) technology can identify and locate faults, monitor the abnormal situation of the entire system, ensure the safety of the production process, and reduce economic losses. [1]. In the past three decades, a large number of process monitoring technologies have emerged, such as model-based monitoring methods [2], structure diagrams, and pattern recognition [3, 4]. However, the process monitoring performance of these methods is still unsatisfactory due to their applicability defects. To crack this dilemma, a number of data-driven methods have recently been exploited [5, 6], which can employ the large amount of data generated by the production process to extract the main information (features) in order to facilitate the prediction or identification of faults. Whereas, the data-driven model is only suitable for interpolation, and once the data is not state ergodic, the effect will be unsatisfactory, especially in the case of multidimensional input.

In recent years, with the flourish of artificial intelligence, FDD based on deep learning [7] has become one of the most promising research hotspots. Due to the training support of massive data, FDD oriented deep network model

✉ Yuwei Ren
yuwei.ren@sdnu.edu.cn

✉ Yixian Fang
fangyixian@qlu.edu.cn

Li Qi
qili96@outlook.com

Jinglin Zhou
jinglinzhou@mail.buct.edu.cn

[1] School of Information Science and Engineering, Shandong Normal University, Jinan 250353, Shandong, China

[2] School of Information Engineering, Shandong Management University, Jinan 250353, Shandong, China

[3] The College of Information Science and Technology, Beijing University of Chemical Technology, Beijing 100029, China

achieves excellent performance in various fault monitoring and diagnosis. Since the initial proposal by Hoskins and Himmelblau, artificial neural networks have been utilized for process monitoring. Researchers have developed mnemonic network models, such as the recurrent neural network (RNN), to tackle sequential problems in fault prediction of process systems [8, 9]. RNN, known for its ability to capture hidden correlations, has found extensive applications in speech conversion, natural language processing, and process fault detection. However, RNN is stuck in the dilemma of gradient disappearance, and its performance gradually decreases with the increase of system variables. Another classical network for sequential fault diagnosis is long short-term neural network (LSTM) [10, 11], which has better time series feature extraction ability in a long time compared with traditional RNN. For example, [12] proposed a fault diagnosis method based on LSTM network, which can adaptively learn dynamic information from the original data and directly classify the original process data without special feature extraction and classifier design. But with the increase of data dimension, its training pressure gradually increases.

To this end, both traditional statistical analysis and current deep learning require feature extraction and dimension reduction of original data to adapt to increasingly complex industrial systems. Traditional dimensionality reduction techniques, such as Kernel Partial Least Squares (KPLS) [13], Kernel Principal Component Analysis (KPCA) [14] and Kernel Canonical Correlation Analysis (KCCA) [15], are only suitable for small data sets with linear correlation and are difficult to adapt to large nonlinear data sets. To face nonlinear dimensionality reduction of big data, an auto-encoder algorithm (AE) [16, 17] based on deep learning is developed, which maps raw data to low-dimensional space through a group of encoders and then reconstructs the mapped data back to the same dimension as the original data through a decoder. Benefiting from the feature extraction and dimension reduction superiorities of AE, scholars have exploited various variant versions for process monitoring. For instance, the proposed de-noising AE (DAE) [18, 19] can achieve a more sensitive and robust monitoring performance than PCA. In order to further reflect the difference of encoding space, another variant of AE [20, 21] implements process monitoring through the nonlinearity and dynamics of probability space. Aiming at the problem of incomplete information extraction in the encoding process [22], manifold regularization stacked auto-encoder (SAE) is proposed to capture the geometric information of data [23, 24]. These research results show that AE algorithm has great advantages in feature extraction. These algorithms usually use the idea of principal component analysis to reduce dimensionality, which only retain the variables that contain the most information. Therefore, in this way, many interactions between variables are ignored.

Formally, the industrial system process can be composed of a group of univariate time series, and each time series represents the attributes and performance of each part in the system. A single variable has a time-dependent characteristic, that is, the variable has regular changes in the same time interval (periodicity) or the mean and variance are the same in each time period. As shown in Fig. 1a, in different periods T, the change breaks the rules of temporal patterns, which is called time dependent failures. Among multiple variables, different variables have their linear relationship. For this fault that violates the historical patterns between variables, we call it inter-variable dependency fault, as shown in Fig. 1b. In the current feature extraction algorithms, these information can not be obtained well.

To overcome the problem that the current deep framework can not consider the relationship between different variables and time series at the same time, we designed a two-view variational LSTM autoencoder to make the fault features more apparent. Concretely, (1) Two sets of LSTM are used to encode the data horizontally and vertically, extract the data variables and time features, and compress them into a low dimensional space for easy interpretation and calculation. (2) Training normal data to obtain normal probability distribution, and judging whether it is abnormal data by calculating anomaly scores. (3) A neuron-grouped convolution layer was added to the extracted feature matrix to classify and train faults. (4) Based on the probability information obtained by fault training, the fault type or unknown fault can be identified to realize fault diagnosis. The main contributions can be summarized as follows:

- A new neuron-grouped two-view variational LSTM auto-encoder is proposed, which preserves both inherent information (the synergistic relationship information among variables in the sample) and temporal information (periodic information between samples of the original data).
- By obtaining the information of TE process, the detection result is superior to the designed algorithm didn't consider this kind of information.
- The convolution of each group is very sensitive to the corresponding fault type, and the fault type can be determined quickly according to the output probability distribution.
- Experimental results show that the proposed fault diagnosis algorithm is significantly superior to the state-of-the-art methods.

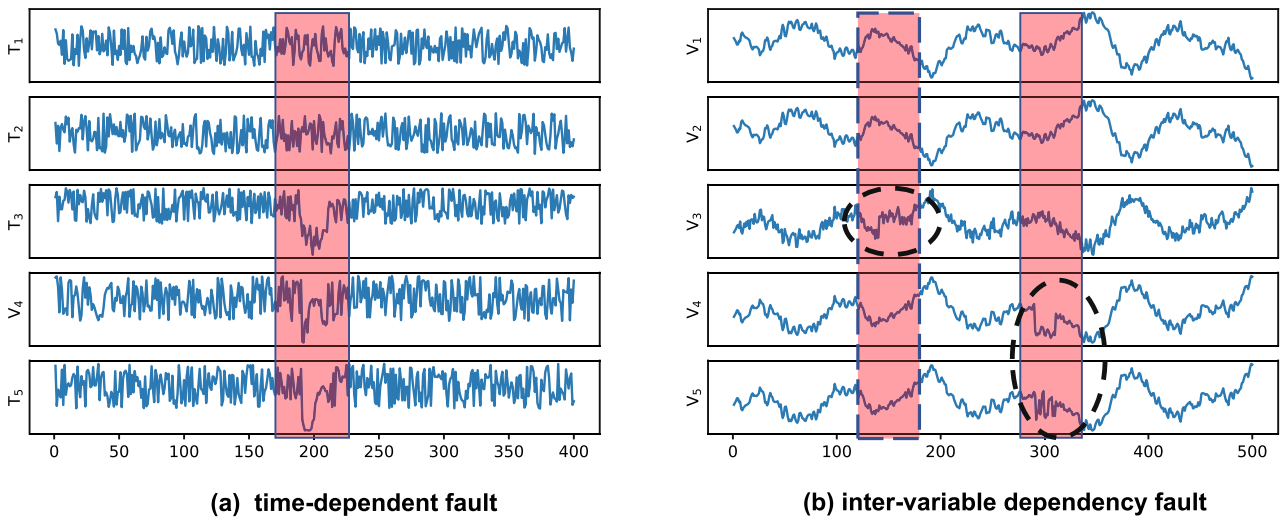

Fig. 1 Description of two types of abnormal data

2 Related works

2.1 LSTM auto-encoder

In the aspect of feature extraction, the traditional data-driven algorithm often selects the eigenvectors corresponding to the larger eigenvalue for dimensionality reduction and feature extraction, which will more or less lose the original data information and ignore the interaction between variables, thus resulting in inaccurate process monitoring. In addition, they are only adapted to linear dimensionality reduction and are unable to cope with nonlinear data. And the neural network has good nonlinear mapping ability and can effectively compress and reduce dimension of data. In particular, LSTM auto-encoders can be used to encode or compress sequence data. In view of the timing of process monitoring, this paper uses it for data reduction and feature extraction. The schematic diagram of auto-encoder is shown in Fig. 2.

Building an auto-encoder involves constructing encoder, decoder and setting a loss function to measure compression loss. The encoder and decoder are generally represented by parametric equations, and the encoder and decoder parameters are determined by optimizing the loss function.

An auto-encoder consists of two cascading networks. The first network is an encoder, encoding input variable x to feature t via function h:

$$t = h(x)$$

The second network uses encoded feature t to reconstruct data r using function f:

$$r = f(y) = f(h(x))$$

The error $e = x - r$ is defined as the difference between the original input x and the reconstructed signal r. Then network training by minimizing errors can realize the purpose of auto-encoder.

The occurrence of faults is often caused by the joint action of multiple variables. LSTM technology can well consider the interaction between different variables. For this purpose, we use LSTM and a full connection layer as an encoder and decoder.

LSTM adds i, f, o, g gate structures on the basis of RNN [8, 9], which makes the whole LSTM have two hidden states h_t and cell state c_t, as shown in Fig. 3.

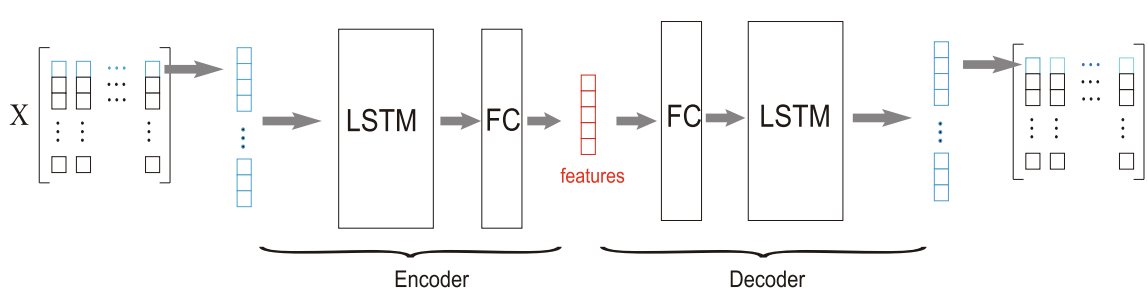

Fig. 2 The schematic diagram of autoencoder

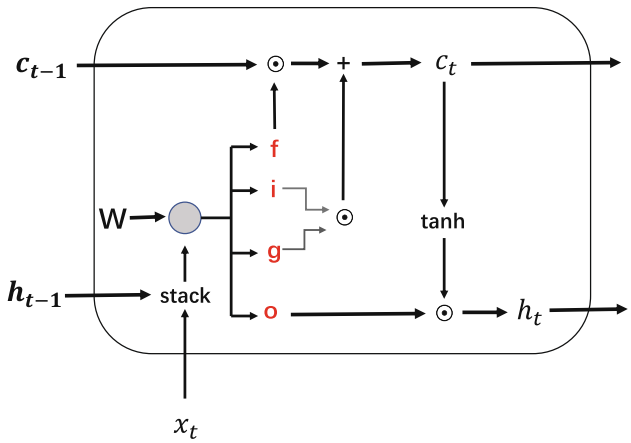

Fig. 3 Schematic diagram of an LSTM neuron

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

Forgetting gate (f) is to decide what to discard from the cell state.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

i and g are update gates that decide to put new information in the cell state.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$g_i = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$
$$C_t = f_t \cdot C_{t-1} + i_t \cdot g_t$$

o is the output gate, which determines the current output state.

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$
$$h_t = o_t \cdot \tanh(C_t)$$

The gate structure of LSTM can well represent the interaction between variables. When the detection value changes, it may result from the influence of several previous variables, rather than all the previous variables.

As shown in Fig. 2, we take the measurement variable $x_t^{1\times m}$ at a certain time as the input and obtain the feature vector $t_t^{1\times l}(l<m)$ through the encoder. Then, the feature vector $t_t^{1\times l}$ passes through the decoder to generate the vector $\hat{x_t}^{1\times m}$. We take the difference between the original input $x_t^{1\times m}$ and the reconstructed variable $\hat{x_t}^{1\times m}$ as the error e. the goal of network training is to reduce the mean square error (MSE) of e. Finally, we get the characteristic matrix T

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(x_t - \hat{x_t})^2 \tag{1}$$

2.2 Monte Carloe estimate

The main idea of Monte Carloe estimate is to obtain the characteristics of a random variable ξ by random sampling, and then calculate its expectations. So, we can give a fault warning based on the expectations of ξ. Specifically, when the probability distribution $p(x)$ is too complex to sample directly, you can define the distribution $q(x)$ (e.g., the Gaussian distribution). So that the probability distribution $q(x)$ can cover $p(x)$, and then the random samples produced by the distribution $q(x)$ are filtered according to a certain method to make the density distribution of the filtered samples $p(x)$. Then, the random number of the distribution $q(x)$ is obtained according to Theorem 2, and is filtered by Algorithm 1, so that the density distribution of the filtered sample is $p(x)$. The specific process is as follows:

(1) Given a Gaussian distribution function $q(x)$, select a suitable constant k so that $kq(x)$ can cover $p(x)$.
(2) A random data z_0 is generated from the density function for $q(x)$, and a random number u is generated from the uniform distribution (0, 1).
(3) If u satisfies the condition $ukq(z_0) \le p(z_0)$, that is, $u \le \frac{p(z_0)}{kq(0)}$, receive z_0, otherwise reject z_0. Repeat this process to get many pseudo-random numbers with a probability density of $p(x)$. The entire Monte Carlo estimation is summarized in Algorithm 1.

Algorithm 1 Monte Carloe estimate

Require: Gaussian distribution function $q(x)$, suitable constant k
Ensure: The data Z of the probability density of $p(x)$.

```
for each z_i in q(x) do
    Draw u in N(0,1);
    if u ≤ p(z_0)/(kq(0)) then
        receive z_i in Z
    else
        reject z_i
    end if
end for
```

If the cumulative distribution function is monotonically increasing, random sampling can be performed on the uniform distribution $U(0, 1)$ according to Theorem 1.

Theorem 1 *If the distribution function $F_X(X)$ of the random variable X is a strictly monotonically increasing continuous function and its inverse function $F_X^{-1}(y)$ exists, then $Y = F_X(X)$ obeys the uniform distribution $U(0, 1)$ on interval* [0, 1]

Proof Since the distribution function $F_X(X)$ only takes values on the interval [0, 1].

When $y<0$, because $F_X(X) = y$ is not possible, so,

$$F_y(y) = P(Y \le y) = P(F_X(X) \le y) = 0$$

When $0 \le y < 1$,

$$\begin{aligned} F_y(y) &= P(Y \le y) = P(F_X(X) \le y) \\ &= P(X \le F_X^{-1}(y)) = F_X(F_X^{-1}(y)) = y \end{aligned}$$

When $y<0$, because $F_X(X) \le y$ is inevitable, so,

$$F_y(y) = P(Y \le y) = P(F_X(X) \le y) = 1$$

To sum up, the distribution function of $Y = F_X(X)$ is

$$Y_Y(y) = \begin{cases} 0 & y<0 \\ y & 0 \le y \le 1 \\ 1 & y \ge 1 \end{cases}$$

This is the same as the uniform distribution function on interval [0, 1], so $Y \sim U(0, 1)$. □

Theorem 2 *The random data obtained by the above sampling methods do obey the target distribution $p(x)$, which means that the distribution functions of them are equal.*

Proof Because z_0 and u are independent of each other.

$$\begin{aligned} P\left(z_0 \le x \| u \le \frac{p(z_0)}{kq(0)}\right) &= \frac{P\left(z_0 \le x, u \le \frac{p(z_0)}{kq(0)}\right)}{u \le \frac{p(z_0)}{kq(0)}} \\ &= \frac{P(z_0 \le x) P\left(u \le \frac{p(z_0)}{kq(0)}\right)}{P\left(u \le \frac{p(z_0)}{kq(0)}\right)} \\ &= \frac{\int_{-\infty}^{x} p(x) dx \frac{p(z_0)}{kq(0)}}{\frac{p(z_0)}{kq(0)}} \\ &= \int_{-\infty}^{x} p(x) \mathrm{d}x \end{aligned}$$

So, this completes the proof. □

3 Two-view LSTM variational auto-encoder

The core idea of TVAE is to extract the fault information and retain the information of original data as much as possible, so as to facilitate the isolation and diagnosis of fault data. The algorithm is divided into two stages: offline training and online detection. In the offline training process, the TVAE model (Sect. 3.1) is trained to obtain the distribution and normal score of normal data. In the online test phase, we use the trained TVAE to get the encoding matrix of the test data, and output the anomaly score through the decoding layer. If the score is higher than the normal score, we will give a fault warning (Sect. 3.2), and train the encoding matrix with its corresponding label through neuron-grouped (Sect. 3.3), so that the fault type is located through the convolution. The algorithm flow is shown in Fig. 4, and each part is described as follows:

3.1 Two-view embedding

Data timing relationship is rich in state change rules and information, which is very important for system process monitoring. However, the current time series data prediction is mostly univariate output, which limits the adaptability and generalization of the system. This is because time series relationships in multivariable process monitoring are difficult to capture. Therefore, this paper uses sliding window to obtain samples, which can effectively solve the above problems. As shown in Fig. 5, set a sliding window with a width of n^*, and the step length of each movement is τ. With each step of the window, a $n^* \times M$ matrix Γ will be obtained, which represents the state of the system in the period n^*. Wherein the rows of the matrix are time series samples and the columns are measurement variables.

In order to obtain the dependence of variables and time in the process of multivariate variables, we designed a dual view encoder. As described in Sect. 2, LSTM has great advantages in processing time-series data because its gate structure can well represent the interaction between variables. For example, when a detection value changes, it may be due caused by the effect of some non adjacent variables, such as x^1, x^4 and x^7. Through training, the relationship between variables can be well obtained.

Specifically, $z_1 = h_1(x^i : M') \in R^{n^* \times d_1}$, where h_1 is the LSTM-encoder layer along the measurement dimension for learning variable embedding z_1. M' is the number of variables. $z_2 = h_2(z_1 : n^{*'}) \in R^{d_2 \times d_1}$. And h_2 is the LSTM-

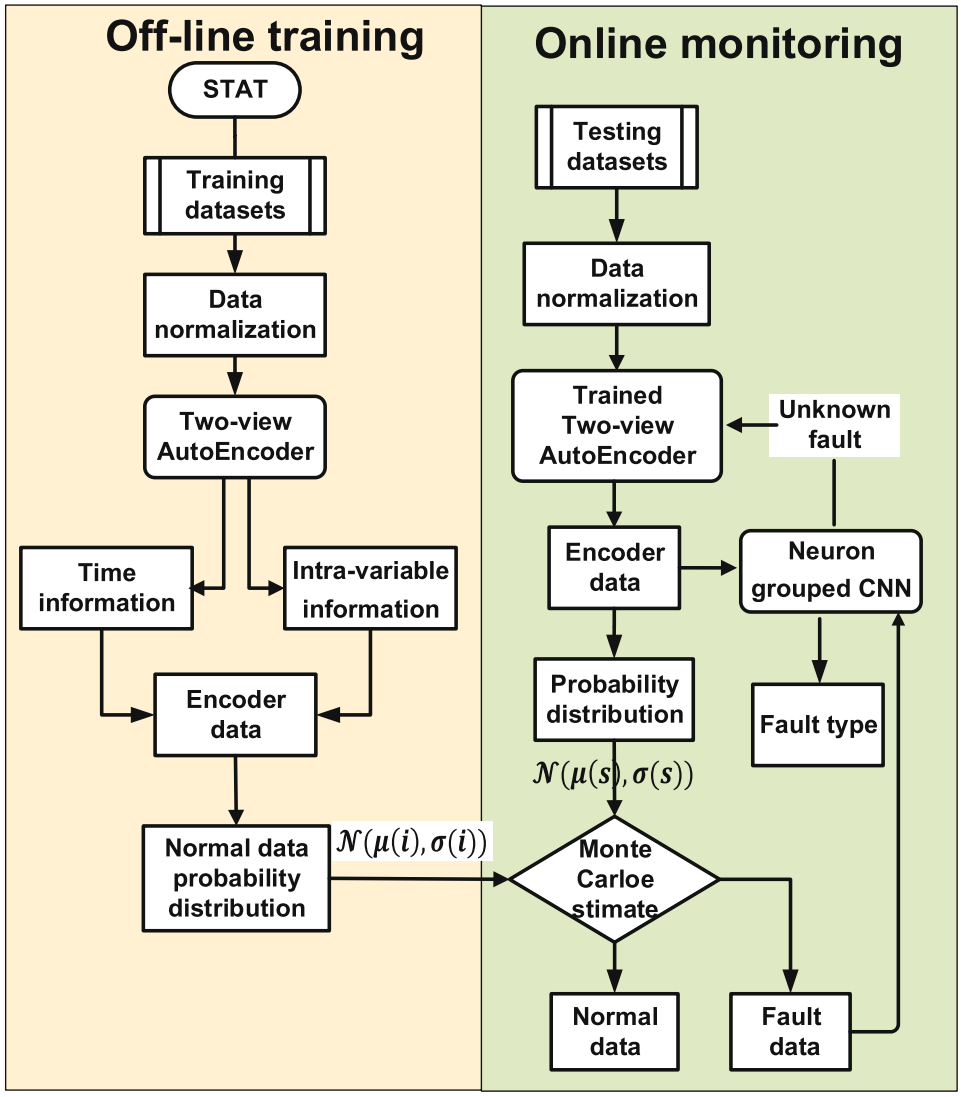

Fig. 4 A flow diagram of TVAE for fault detection and diagnosis

encoder layer along the time dimension applied on z_1 for getting timing information. Finally, the encoding matrix is weighted by two LSTM outputs.

$$Z = \alpha z_1 + \beta z_2$$

where α and β are hyper parameters used to balance the inherent and temporal information with $1 - \alpha$. So that we map the original data from the high-dimensional space to the low-dimensional space while preserving the temporal and intermetric information of the original data.

It is worth emphasizing that this paper is only based on the characteristics of the fault to obtain more targeted feature, rather than to extract as much information as possible from the original data. The targeted feature information is very important for fault identification and diagnosis, and its value is much greater than the impact of increased noise.

3.2 Two-view embedding variational auto-encoder for fault detection

At present, AE based anomaly detection methods use reconstruction errors as outliers, but if the input variables are heterogeneous, it is difficult to calculate outliers. Therefore, in order to summarize the differences of heterogeneous data, it is necessary to carry out weighted summation. However, there is no objective method to determine the appropriate weight, because the weight will vary from data to data. Moreover, it is troublesome to determine the threshold of the reconstruction error even after the weight is determined. There is no clear explanation, so the encoding matrix obtained by AE lacks generalization ability and is less robust in fault feature detection. Considering the variability of data and making the abnormal data more interpretable, this paper uses a variational autoencoder (VAE) to identify the fault features.

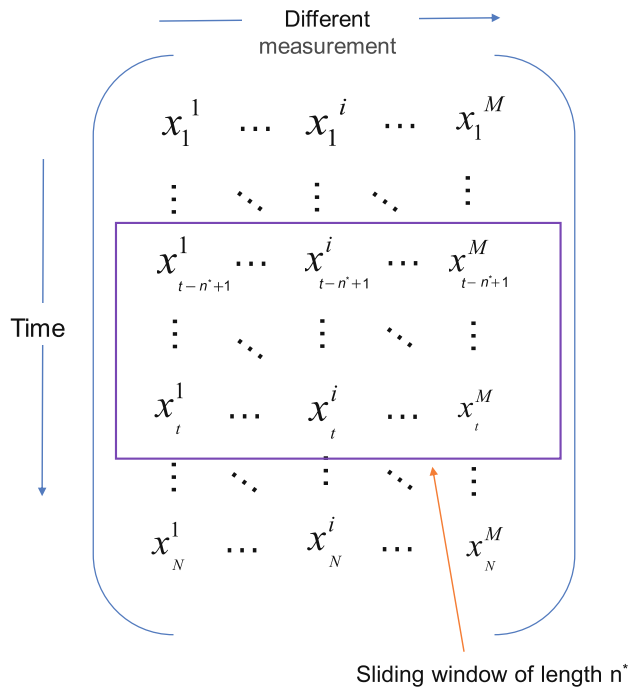

Fig. 5 Feature extraction at a certain time by sliding window

The core idea of the VAE is to first suppose the distribution of a encoding matrix Z and build a model from Z to the target data X', that is, to build $X' = g(Z)$, so that the probability distribution of the learned target data X' is similar to that of the real data X. That is, VAE uses the encoder and decoder to fit the mean and variance. In addition, as shown in theorem 1, VAE makes each Gaussian distribution approach the standard Gaussian distribution $N(0, 1)$ as much as possible. The error loss in this fitting process is calculated by using KL divergence. The following is a detailed derivation:

The probability distribution of the original sample data X is

$$P(x) = \int_Z P(z)P(x|z)\mathrm{d}z$$

Supposing that Z obeys the standard Gaussian distribution,The prior distribution $P(x|z)$ is also a Gaussian distribution, that is, $P(X|Z) \sim N(\mu(z), \sigma(z))$. Since $P(z)$ is known and $P(x|z)$ is unknown, solving the problem is actually finding μ, σ. The goal is to maximize $P(x)$, which is equivalent to solving the maximum likelihood number about x

$$L = \sum_x \log P(x)$$

According to Theorem 2, $\log P(x)$ can be calculated by:

$$\begin{aligned}
\log P(x) &= \int_Z P(z|x) \log P(x)\mathrm{d}z \\
&= \int_Z P(z|x) \log P\left(\frac{P(z,x)}{P(z|x)}\right)\mathrm{d}z \\
&= \int_Z P(z|x) \log P\left(\frac{P(z,x)}{q(z|x)}\frac{q(z|x)}{P(z|x)}\right)\mathrm{d}z \\
&= \int_Z P(z|x) \log P\left(\frac{P(z,x)}{q(z|x)}\right)\mathrm{d}z \\
&+ \int_Z P(z|x \log P\left(\frac{q(z,x)}{P(z|x)}\right)\mathrm{d}z \\
&= \int_Z P(z|x) \log P\left(\frac{P(x|z)P(z)}{q(z|x)}\right)\mathrm{d}z \\
&+ \int_Z P(z|x) \log P\left(\frac{q(z|x)}{P(z|x)}\right)\mathrm{d}z
\end{aligned}$$

We can find that $\int_Z P(z|x) \log P(\frac{q(z|x)}{P(z|x)})\mathrm{d}z$ is the KL divergence of q and P, that is,$KL(q(z|x), P(z|x))$. Because KL divergence is greater than or equal to 0, we can find the lower bound of $\log P(x)$

$$\log P(x) \leq \int_Z P(z|x) \log P\left(\frac{P(x|z)P(z)}{q(z|x)}\right)\mathrm{d}z$$

Let

$$L_b = \int_Z P(z|x) \log P\left(\frac{P(x|z)P(z)}{q(z|x)}\right)\mathrm{d}z$$

so,

$$\log P(x) = L_b + KL(q(z|x)||P(z|x))$$

In order to find the maximum of $\log P(x)$, it can be converted to maximize this lower bound L_b.

Theorem 3 *Maximizing* $\log P(x)$ *is equivalent to maximizing* L_b

Proof Because the posterior distribution P is intractable, it is used to approximate the posterior distribution with $q(z|x)$. $q(z|x)$ and $\log P(X)$ are independent of each other. $\log P(x)$ is only related to $P(z|x)$. adjusting $q(z|x)$ will not affect $\log P(x)$. Therefore, when $P(x|z)$ is fixed, the adjustment $q(z|x)$ maximizes the lower bound L_b, and KL will be smaller. When $q(z|x)$ keeps approaching the posterior distribution $P(x|z)$, KL divergence tends to be 0, and $\log P(x)$ is equivalent to L_b. So maximization is equivalent to maximization, this completes the proof. □

According to the Expectation Maximization algorithm, we get

$$
\begin{aligned}
L_b &= \int_Z q(z|x)\log P\left(\frac{P(x|z)P(z)}{q(z|x)}\right)\mathrm{d}z \\
&= \int_Z q(z|x)\log P\left(\frac{P(z)}{q(z|x)}\right)\mathrm{d}z + \int_Z q(z|x)\log P(P(x|z))\mathrm{d}z \\
&= -KL(q(z|x)||P(z)) + \int_Z q(z|x)\log P(P(x|z))\mathrm{d}z \\
&= -KL(q(z|x)||P(z)) + E_{q(z|x)}[\log P(x|z)]
\end{aligned}
$$

thus maximizing L_b is equivalent to minimizing $KL(q(z|x)||P(z))$ and maximizing $E_{q(z|x)}[\log P(x|z)]$.

Minimize KL divergence: It has been supposed that $P(Z)$ obeys the standard Gaussian distribution, and $q(z|x)$ obeys the Gaussian distribution $N(\mu, \sigma^2)$, so it can be substituted into the calculation:

$$
\begin{aligned}
&KL(q(z|x)||P(z)) = KL(N(\mu,\sigma^2)||N(0,1)) \\
&= \int \frac{1}{\sqrt{2\pi\sigma^2}} e^{\frac{-(x-\mu)^2}{2\sigma^2}} \left(\log \frac{e^{\frac{-(x-\mu)^2}{2\sigma^2}}/\sqrt{2\pi\sigma^2}}{e^{\frac{-x^2}{2}}/\sqrt{2\pi}}\right)\mathrm{d}x \\
&= \frac{1}{2}\int \frac{1}{\sqrt{2\pi\sigma^2}} e^{\frac{-(x-\mu)^2}{2\sigma^2}} \\
&\quad \left(-\log\sigma^2 + x^2 - \frac{-(x-\mu)^2}{2\sigma^2}\right)\mathrm{d}x \\
&= \frac{1}{2}\left(-\log\sigma^2 + \mu^2 + \sigma^2 - 1\right)
\end{aligned}
$$

Maximize expectations $\mathrm{MAX}(E_{q(z|x)}[\log P(x|z))$ means that given the encoder output $q(x|z)$, the mean and variance of the decoder output $P(x|z)$ are as close as possible

In the first step, the encoder's neural network is used to calculate the mean and variance, and z is sampled from them. This process corresponds to $q(x|z)$.

The second step is to use the mean and variance of the decoder's output, so that the closer the mean and variance is to X, corresponding to the maximization $\log P(x|z)$

For the online fault detection process, we use the $E_{q(z|x)}[\log P(x|z)]$ as the anomaly score. However, anomalies in the test data may lead to bias in learning embedding and affect the estimation of reconstruction [25]. We assume that all the new data X_{new} are fault data, and use MC to sample a large number of them. The anomaly score can be solved by Monte Carlo estimation, sampling S points from $q(z|x)$ according to the probability distribution of Z, that is

$$
E_{q(z|x)}[\log\ P(x|z)] \approx \frac{1}{S}\sum_{s=1}^{S}\log\ p(x|z^s)
$$

But this operation is non-differentiable. it can use parameterization techniques to make the sampling operation differentiable. We suppose that $z^s = g(x, \varepsilon^s)$, $\varepsilon^s \approx p(\varepsilon)$, which ε^s and $p(\varepsilon)$ are known. So,

$$
E_{q(z|x)}[\log P(x|z)] \approx \frac{1}{S}\sum_{s=1}^{S}\log p(x|g(x,\varepsilon^s))
$$

In this way, we can find the anomaly score. A high score indicates that the system deviates from the normal mode. The entire fault monitoring process is summarized as Algorithm 2.

Algorithm 2 TVAE for fault detection

Require: Normal data X_{normal}, Fault data $X_{new} \in R^{N\times M}$. Sliding window $W \in R^{n^*\times M}$

Ensure: Anomaly score.

```
Training TVAE model to obtain probability encoding layer p and normal score α
Obtain the test input matrix in n* time interval by sliding the window W_test
for i = 1 to N/n* do
    μ_i, σ_i = p(z|w^i_test)
    Draw S samples from z ~ N(μ_i, σ_i);
    for i = 1 to S do
        μ̂_{i,s}, σ̂_{i,s} = g(x|z^{i,s})
    end for
    Anomaly score=(1/S) Σ_{s=1}^{S} log p(w^i_test|μ̂_{i,s}, σ̂_{i,s})
    if Anomaly score ≤ α then
        w^i_test is fault data
    else
        w^i_test is norm data
    end if
end for
```

3.3 Neuron-grouped TVAE for fault classification

The vertical movement of convolution kernel can obtain the time sequence information of the original characteristic matrix, and the horizontal movement of convolution kernel can also obtain the information between variables. Some features are learned through the movement of convolution kernel, and then applied to any position of Encoded matrix Z, so as to obtain the activation value of a different feature at any position on matrix Z, such as Fig. 6:

$$v_1^* = \sum v_{ij} d_{ij}$$

where v_{ij} is the value of row i and column j of matrix Z, and d_{ij} is the parameter of row i and column j of convolution kernel. In Fig. 6, only one convolution kernel is described and only one feature is learned, which is obviously insufficient. We can add multiple convolution kernels. For example, 32 convolution kernels can learn 32 features $w = [d_{ij}^1, d_{ij}^2, \ldots, d_{ij}^{32}]$ to make process monitoring more accurate.

After obtaining the features through convolution, we hope to use these features for fault recognition. With the increase of convolution depth, the eigenvalues obtained by convolution increase sharply. Theoretically, all extracted features are required to train classifiers, such as Softmax classifier, but computational challenges are faced. It is very inconvenient to learn a classifier with tens of thousands of feature inputs, because it is prone to over fitting. In order to solve this problem, a natural idea is to aggregate the features. For example, the average value or maximum value of a specific feature in a region of the matrix can be calculated. These summary statistical features not only have much lower dimensions, but it is also improved the accuracy of the results.

The goal of convolution is to learn the category characteristics of different faults in order to improve the accuracy of recognition. However, unsupervised training often extracts some features unrelated to fault, which reduces the accuracy of fault classification. Therefore, we add fault label information in the training process to obtain the characteristics of relevant faults. In the convolution process, the characteristics of a certain type of fault are learned through the number of convolution kernels, so we divide the convolution kernels into L groups. L is the number of fault types in historical data, and each group of convolution kernels is used to train the characteristics of a single fault. The original data is convoluted by multi-layer convolution kernel, and finally output L probability values $[P_1, P_2, \ldots, P_L]$ through full connection layer and softmax, as shown in Fig. 7.

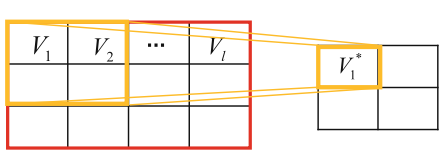

Fig. 6 A flow diagram of CNN

$$p_i = \frac{e^{h_i}}{\sum_{k=1}^{L} e^{h_k}} \tag{2}$$

where h is the output of the full connection layer. The output probability is calculated by cross entropy with the one-hot label of the original fault label.

$$J = -\sum_{i=1}^{L} l_i \log(p_i) \tag{3}$$

The one-hot label of the cth category is $[0, \ldots, 1_c, \ldots, 0]^{\mathrm{T}}$, The cth element is 1, and other elements are 0. Through the training of cross entropy loss function, the characteristics of each type of fault are more obvious, so that the distance between classes of different faults is expanded and the distance within classes is reduced, so that the extracted features are highly separable. The training process is shown in Fig. 8

Theorem 4 *For classification problems, softmax with cross entropy loss function has more advantages than softmax with MSE combination.*

Proof The cross entropy loss function and softmax used in this paper are given in Eqs. (2), (3). The derivation process is as follows.

$$\frac{\partial J}{\partial h} = \sum_j \left(\frac{\partial J_j}{\partial p_j} \frac{\partial p_j}{\partial h_j} \right)$$

The reason why we choose to use the subscript j instead of i is that because the denominator of softmax contains all the outputs of neurons. Therefore, other outputs that are not equal to i also contain p_i. All p should be included in the calculation range, and the subsequent calculation also needs to be divided into two cases: $i = j$ and $i \neq j$.

$$\frac{\partial J_j}{\partial p_j} = \frac{\partial(-l_j \log(p_j))}{\partial p_j} = -l_j \frac{1}{p_j}$$

if $i = j$:

$$\begin{aligned} \frac{\partial p_i}{\partial h_i} &= \frac{\partial \frac{e^{h_i}}{\sum_{k=1}^{L} e^{h_k}}}{\partial h_j} = \frac{\sum_k e^{h_k} e^{h_i} - (e^{h_i})^2}{\left(\sum_k e^{h_k}\right)^2} \\ &= \left(\frac{e^{h_i}}{\sum_k e^{h_k}} \right) \left(1 - \frac{e^{h_i}}{\sum_k e^{h_k}} \right) \\ &= p_i(1 - p_i) \end{aligned}$$

if $i \neq j$:

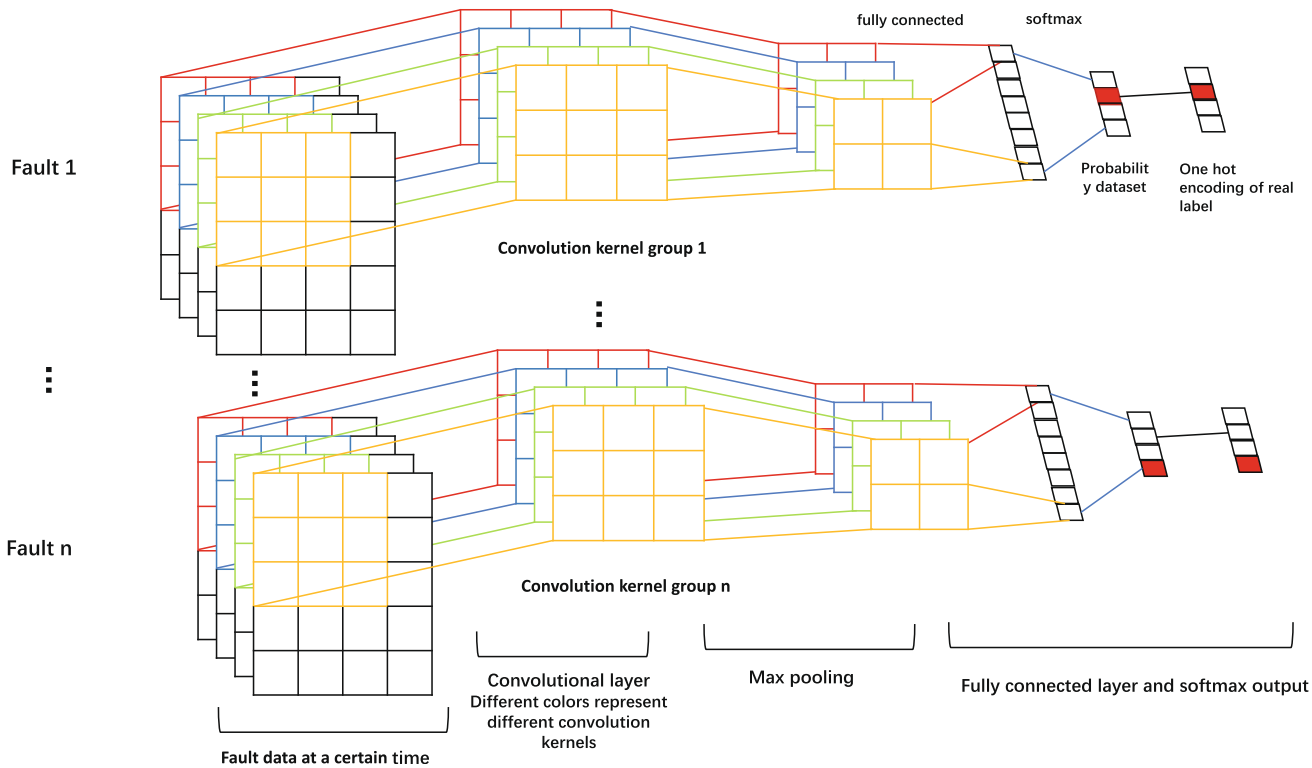

Fig. 7 Neuron grouped training process

$$\frac{\partial p_i}{\partial h_i} = \frac{\partial \frac{e^{h_i}}{\sum_{k=1}^{L} e^{h_k}}}{\partial h_j} = -e^{h_j}\left(\frac{1}{\sum_k e^{h_k}}\right)^2 e^{h_i} = -p_i p_j$$

Then, merge the above formula:

$$\begin{aligned}\frac{\partial J}{\partial h} &= \sum_j \left(\frac{\partial J_j}{\partial p_j}\frac{\partial p_j}{\partial h_j}\right) = \sum_{j\neq i}\left(\frac{\partial J_j}{\partial p_j}\frac{\partial p_j}{\partial h_j}\right) + \sum_{i=j}\left(\frac{\partial J_j}{\partial p_j}\frac{\partial p_j}{\partial h_j}\right)\\ &= \sum_{j\neq i} -l_j\frac{1}{p_j}(-p_i p_j) + \left(-l_i\frac{1}{p_i}\right)(p_i(1-p_i))\\ &= \sum_{j\neq i} p_i y_i + (-l_i(1-p_i))\\ &= \sum_{j\neq i} p_i l_i + a_i l_i - l_i\\ &= p_i\sum_j l_j - l_i\end{aligned}$$

However, the MSE loss function is $loss = \sum_{i=1}^{n}(y_i - \hat{y_i})^2$,where $\hat{y} = \sigma(w^{\mathrm{T}}x_i)$

$$\frac{\partial \mathrm{loss}}{\partial w} = \sum_{i=1}^{n}(-2(y_i - \sigma(w^{\mathrm{T}}x_i))\sigma(w^{\mathrm{T}}x_i)(1-\sigma(w^{\mathrm{T}}x_i))x_i)$$

Because the goal of MSE gradient descent is to make $\sigma(w^{\mathrm{T}}x_i)$ or $(1-\sigma(w^{\mathrm{T}}x_i))$ close to 0, there will be the problem of gradient disappearance. Softmax with cross entropy will not have this problem. So, this completes the proof. □

For unknown types of faults, we set two thresholds $\theta_{\max}$ and θ_{var} in the process of online monitoring, where $\theta_{\max}$ is the threshold of maximum probability value and θ_{var} is the threshold corresponding to probability variance.

$$\begin{cases}\text{Unknown fault} & \max\ p_i < \theta_{\max} \text{ or } \mathrm{var}\ p_i < \theta_{\mathrm{var}}\\ \text{fault type} & \text{others}\end{cases}$$

Finally, we can obtain feature information of different faults through several sets of convolution kernels and fault labels. At the same time, the threshold can be calculated from the variance and probability distribution of the network output.

3.4 Algorithm performance

We evaluated the performance and computational pressure of each algorithm through time complexity. By comparing the principles of [26–29], we can see that their main differences lie in the expansion of the coding process.

For the integration of RACAE [26] using convolution kernel and AE, it is assumed that the convolution kernel size is $H * W$, the input channel is I, and the output channel is O. The total number of filter parameters in the Convolutional Layer is $H * W * I$, and the calculation of each filter requires one bias, so the total number of parameters is $(H * W * I + 1) * O$. In practical applications, the setting of the value H, W and I is very small.

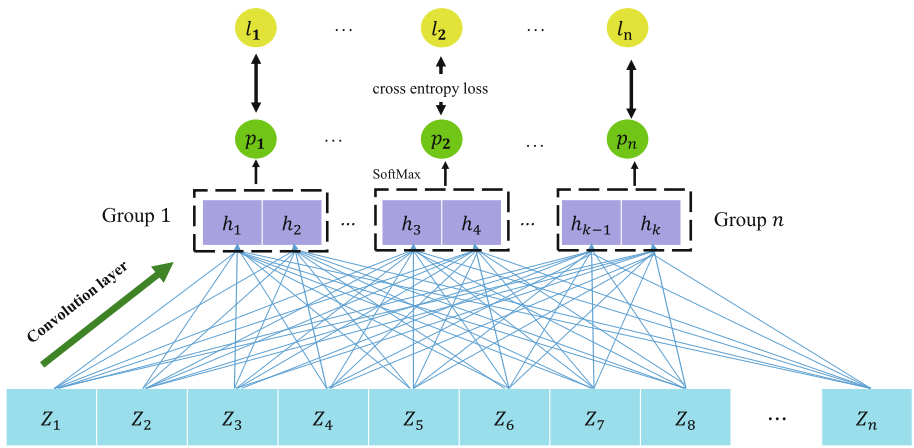

Fig. 8 Training process diagram in Neuron grouped

For NCAA [27] fully connected neural network, there are n-dimensional inputs and m-dimensional outputs before and after each layer, so the number of parameters is $(n+1)*m$.

For the LSTM used in this paper, we can simplify the parameters into two matrices, U and V, which map the input and output respectively. The dimension of U is hidden * Input, and the dimension of V is hidden * hidden. Because LSTM has four gate structures, the total number of parameters is 4 (hidden * Input + hidden * hidden + hidden)

From the above analysis, it can be concluded that the parameters of fully connected neural network are the least and LSTM the most. However, for RACAE [26] and NCAA [27] both use deep network structure, this paper only uses one LSTM layer and one full connection layer, as shown in Fig. 2. So, our algorithm is suitable for shallow networks.

4 Case study

Tennessee Eastman (TE) process [30] is an industry benchmark of process monitoring, consisting of five main reactor units: condensers, compressors, separators, stripper and eight components of $A \sim H$, based on data generated from real industrial processes. It consists of 53 variables, including 12 control variables $[\mathrm{XMV}(1)\cdots\mathrm{XMV}(12)]$ and 41 measurement variables $[\mathrm{XMEAS}(1)\cdots\mathrm{XMEAS}(41)$, where the $\mathrm{XMEAS}(1)\sim\mathrm{XMEAS}(22)$ is sampled every 3 min, the component measurement variable $\mathrm{XMEAS}(23)\sim\mathrm{XMEAS}(36)$ is sampled every 6 min and the remaining $\mathrm{XMEAS}(37)\sim\mathrm{XMEAS}(41)$ is sampled every 15 min. All measured values contain Gaussian noise. TE process has a common data set, which includes a set of normal fault data sets (IDV (0)) and 21 sets of fault data set IDV(1) $\sim$ IDV(21). Table 1 is the specific description of its fault type. The training set contains 500 observations, while the test set contains 960 observations. The fault was introduced at the 161st time. To facilitate the comparison of results, this paper selects 33 measurement variables following [31], including 22 process measurements and 11 manipulated variables.

Table 1 The fault description and fault type of TE process

Fault ID	Fault description	Type
1	A/C feed ratio, B constant	Step
2	B composition, A/C ratio constant	Step
3	D feed temperature	Step
4	Reactor water inlet temperature	Step
5	Condenser water inlet temperature	Step
6	A feed lost	Step
7	C header pressure loss	Step
8	A, B, C feed composition	Random
9	D feed temperature	Random
10	C feed temperature	Random
11	Reactor water inlet temperature	Random
12	Condenser inlet temperature	Random
13	Reaction kinetics	Slow drift
14	Reactor cooling water valve	Sticking
15	Condenser cooling water valve	Sticking
16-20	Unknown	Unknown
21	Valve fixed in steady position	Constant position

In order to simulate the industrial process, the TAVE model is successively input according to the sequence relation of the original data. The sliding window is set as 20×33 and the moving step is 1. In addition, the number of nodes of each LSTM hidden layer is set to 200, and that of the full connection layer is set to 16, and the maximum number of iterations is set to 100.

First, we use normal data to train TVAE to obtain a normal score α. And then use the fault data as the test data.

The anomaly score output by the decoder layer is obtained through the trained TVAE model. Figure 9 shows the test result of fault 1, where the abscissa is the time and the ordinate is the anomaly score. The red dashed line in the figure indicates the normal score (threshold) α obtained in the training process. It can be seen that almost all the normal data are below the threshold. After the fault is introduced at time 160, most of the abnormal data are above the normal threshold, which proves that the validity of fault identification.

Similarly, we take the remaining abnormal data as test data, and Fig. 10 shows the other types of remaining faults. All fault detection rates (FDR) are compared with the latest algorithms [26, 27], and the comparison results are shown in Table 2. The comparison results in all the tables below, with the bold values indicating the best results.

$$\mathrm{FDR} = \frac{\text{Number of effective alarms}}{\text{Total faulty samples}} \times 100\%$$

It can be seen from Table 2 that most of the algorithms are better than the latest research results. It is worth noting that the performance of the algorithm for fault 5, fault 9 and fault 15 is more excellent. Fault 5 is caused by the temperature change of condensate wate. Fault 9 is the temperature of D feed and fault 15 is the viscosity of condenser cooling water valve. They are all related to the temperature of the system. The temperature of the system will affect most of the measured value changes. LSTM can get the relationship between variables, which can capture the characteristics of synergistic changes among variables well. In statistical analysis algorithms, principal components are often used to reduce the dimension, which ignore the information of synergistic changes among variables,

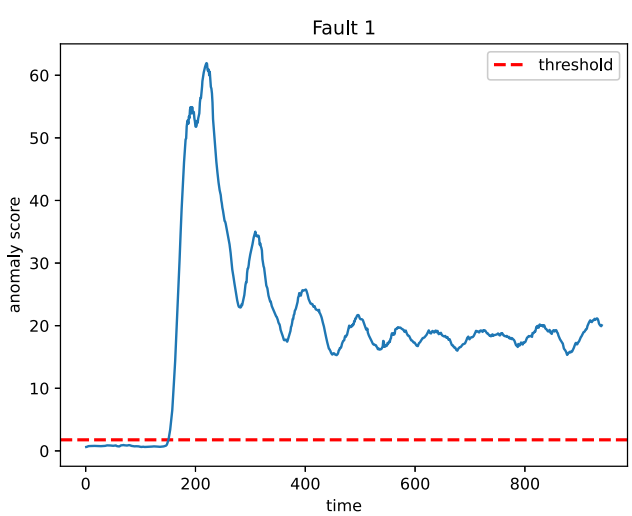

Fig. 9 TVAE fault detection results of fault 1

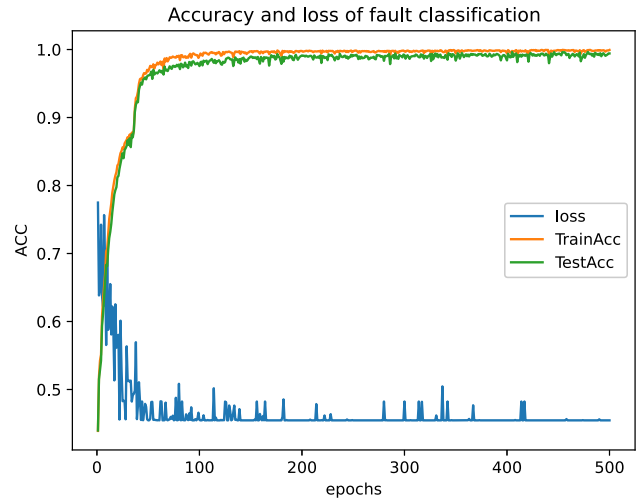

Fig. 10 TVAE fault detection results of some other fault types

Table 2 Comparison fault detection rate of different methods

	KPCA	NCA	SDAE	RACAE	TVAE
Fault 1	99.75	99.65	99	**100**	**100**
Fault 2	98.75	99.52	98	99	**100**
Fault 3	9.45	22.11	2	13	**38.3**
Fault 4	**100**	95	99.85	**100**	**100**
Fault 5	5.8	25.30	34	73	**93.29**
Fault 6	99.9	99.58	**100**	**100**	**100**
Fault 7	**100**	**100**	**100**	**100**	**100**
Fault 8	97.50	97.75	99	99.1	**100**
Fault 9	12.25	21.86	15	13	**52.78**
Fault 10	85.50	87.20	73	82	**91.65**
Fault 11	98.10	98.20	76	80	**100**
Fault 12	31.15	42.06	99	**100**	**100**
Fault 13	94.95	94.46	96	**96**	95.94
Fault 14	99.0	99.75	100	**100**	**100**
Fault 15	6.20	19.36	24	17	**67.56**
Fault 16	5.45	12.85	**70**	64	61.23
Fault 17	92.25	94.22	95	96	**98.24**
Fault 18	70.3	75	91	92	**96.38**
Fault 19	99.0	98.55	35	74	**99.25**
Fault 20	87.3	85.43	76	79	**90.9**
Average	69.68	73.63	74.09	83.25	**89.72**

which reduces the sensitivity to some faults, resulting in large differences in results. Our experimental results further confirm that the algorithm can retain more fault information.

Separate the fault data through TVAE algorithm, and detect all possible fault data. We want to diagnose the separated faults so that system problems can be quickly

resolved. To this end, we use the encoded matrix of all faults for model training, including 20 fault types with their labels for training and one fault type for testing.

Different from a network structure corresponding to one type of faults, all faults adopt the same network structure, and the same number of convolution kernels are allocated to each type of faults, and the same pool layer and full connection layer are used to facilitate the design of the model. For example, the first layer of a set of fault data requires 16 convolution kernels of size 3×1. Since there are 20 fault types, a total of 20×16 convolutional kernels are required. The network structure is shown in Table 3.

Table 3 The network structure used in CNN groups

Layer Name	One grpup CNN operator	Total CNN operator
Layer 1	Conv(16,3,1)	Conv(320,3,1)
Layer 2	Conv(8,3,1)	Conv(160,3,1)
Layer 3	MaxPool(2,2)	MaxPool(2,2)
Layer 4	Conv(6,3,1)	Conv(120,3,1)
Layer 5	MaxPool(2,1)	MaxPool(2,1)
Layer 6	FC($3 \times 5 \times 120$,200)	
Layer 7	FC(200,20)	
Layer 8	FC(200,20)	
Layer 9	Softmax classifier	

We take 80% of each group of fault data as training data and the rest as test data. The results are shown in Fig. 11, where the abscissa is training times (500 epoch) and the ordinate is precision and loss value. The results in Fig. 11 show that the classification accuracy of the algorithm is close to 100%.

For each type of fault, we use the Recall, false positive rate (FPR) to measure the performance of each type of diagnosis, which are defined as:

$$\begin{aligned} \text{Recall} &= \frac{\text{TP}}{\text{TP}+\text{FP}} \\ \text{FPR} &= \frac{\text{FN}}{\text{FN}+\text{TN}} \end{aligned} \tag{4}$$

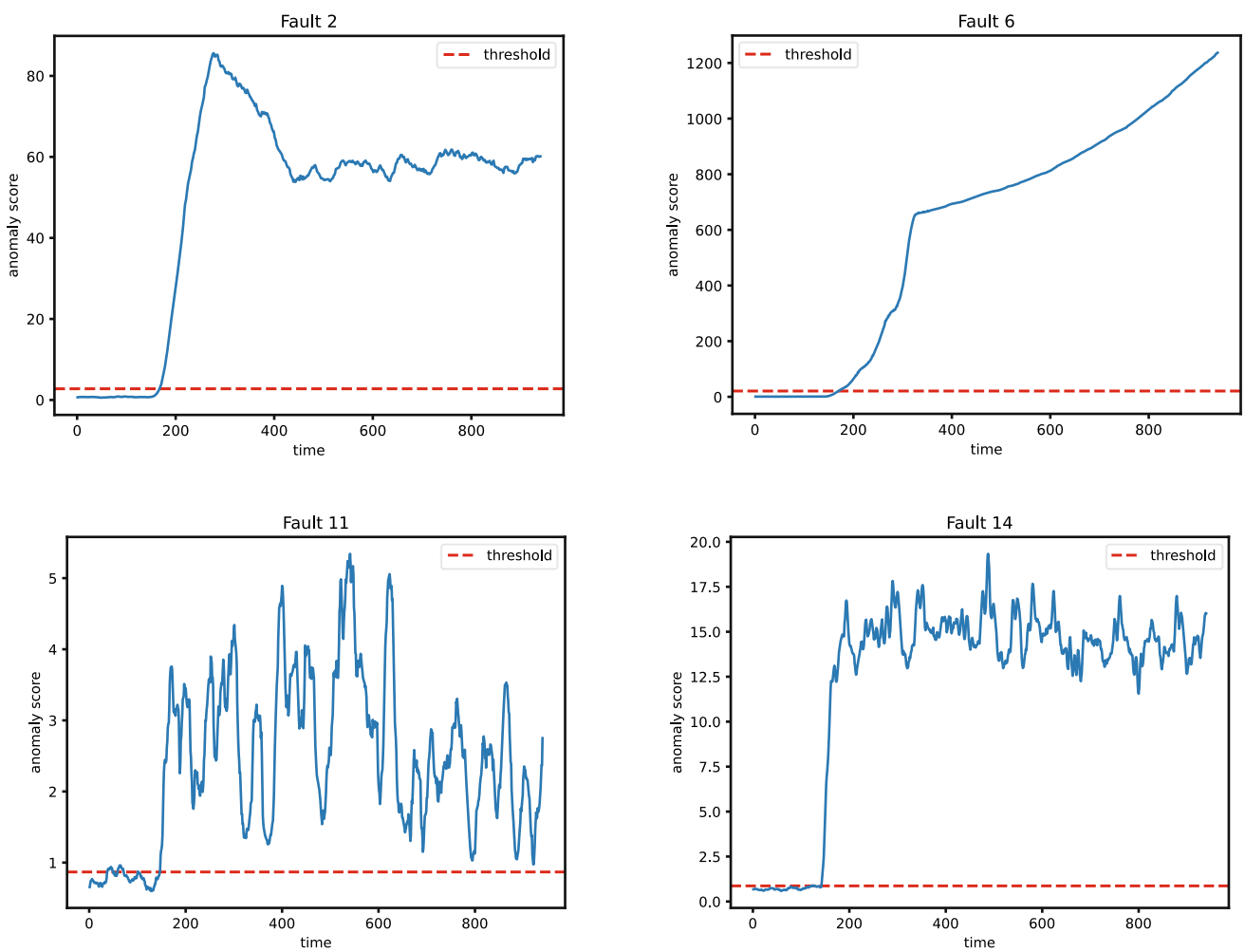

Fig. 11 Accuracy and loss of fault classification

where TP, FP, FN, TN is defined in Table 4.

To verify the advance of the proposed method, two popular fault classification algorithms are selected for comparison [31, 32]. As shown in Table 5, in most types of fault diagnosis, the algorithm proposed in this paper leads both in fault diagnosis rate and fault classification accuracy. Moreover, it has low false alarm rate and is suitable for fault diagnosis of industrial process.

Figure 12 is the confusion matrix of fault type prediction. The abscissa represents the predicted fault type and the ordinate represents the real fault sample. The numerical value and color represent the prediction results, which show the superiority of the proposed method.

In other relevant studies, faults of TE process are divided into three tested cases [33]:

Case 1: Reactor coolant temperature step change and reactor kinetics slow drift.

Case 2: A Sudden loss of feed Condenser cooling water inlet temperature and reaction kinetics slow drift.

Case 3: Step change of reactor cooling water inlet temperature, D Random change of feed temperature and random change of reactor cooling water inlet temperature.

To further verify the performance of TVAE, fault diagnosis is also carried out under the same three conditions, and the comparison of its accuracy is shown in Tables 6, 7 and 8.

By comparing the results of the three cases, we can see that TVAE improves by 2.04%, 2.67% and 10.24% in three cases respectively, which also shows that the algorithm has good applicability. Figure 13 shows TSNE visualization of classification results for three scenarios.

For unknown faults, we analyze the maximum probability of softmax layer and the data distribution of full connection layer. We use the K-fold cross-validation method to divide the data set into 21 parts, 20 of which are used as training sets, and the remaining one as validation sets. We perform 100 rounds in this way to obtain 100 evaluation results. Select the probability and variance corresponding to the best model as the thresholds. Therefore, we define two thresholds: the maximum probability threshold is 0.9 and the minimum variance threshold is 40. If the maximum probability of fault output is less than 0.9 or the variance is less than 40, it is considered as an

Table 4 Recall and FPR parameter description

Layer name	Class = L (predicted)	Class $\neq L$ (predicted)
Class = L (actual)	TP	FP
Class $\neq L$ (actual)	FN	TN

Table 5 Comparison of Recall and FPR of different methods

Fault type	Recall (%)			FPR (%)		
	TVAE	GA-DCNN	CG-SAE-4	TVAE	GA-DCNN	CG-SAE-4
Fault 1	**100**	99.67	**100**	**0**	0.14	**0**
Fault 2	**100**	99.52	**100**	**0**	0.01	**0**
Fault 3	**100**	99.52	–	0.25	0.65	–
Fault 4	98.71	99.92	**100**	2.43	0.04	**0**
Fault 5	97.43	99.72	**100**	0.89	0.39	**0**
Fault 6	**100**	99.29	98.82	**0**	**0**	0.006
Fault 7	**100**	99.95	**100**	0.12	**0**	**0**
Fault 8	**100**	99.97	96.85	2.3	**0.10**	0.15
Fault 9	**100**	97.69	–	**0.76**	0.88	–
Fault 10	**97.94**	97.75	89.36	**0**	**0**	0.51
Fault 11	**99.87**	99.45	**99.87**	**0**	0.03	0.01
Fault 12	**99.74**	99.36	**99.74**	**0**	0.16	0.01
Fault 13	**99.74**	97.19	91.46	0.26	**0.01**	0.41
Fault 14	**100**	99.77	**100**	**0**	**0**	**0**
Fault 15	99.74	**99.77**	–	**0**	3.92	–
Fault 16	**99.61**	90.56	93.17	**0**	3.95	0.32
Fault 17	**100**	97.59	**100**	**0**	**0**	**0**
Fault 18	**100**	96.45	93.82	**0**	0.01	0.29
Fault 19	**99.74**	99.16	92.64	0.26	**0**	0.35
Fault 20	**99.87**	96.81	94.22	0.12	**0.04**	0.27
Average	**99.61**	98.36	96.02	**0.37**	0.48	3.98

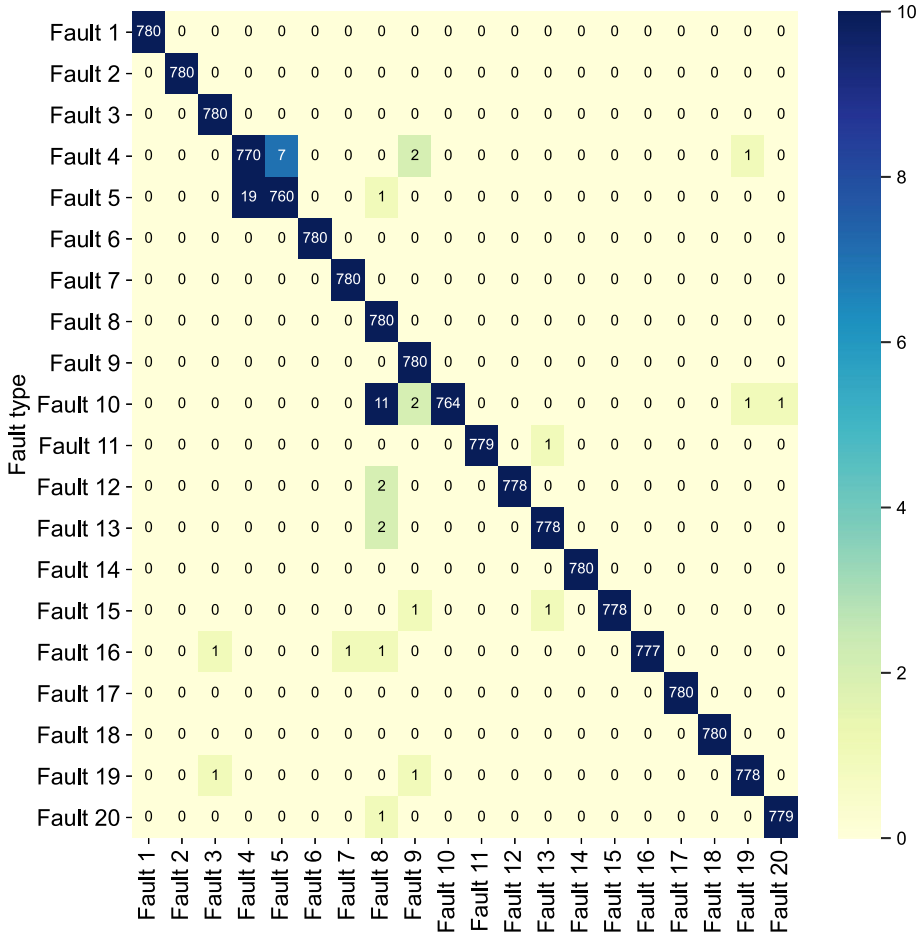

Fig. 12 Details of fault type prediction results

Table 6 Diagnosis results of TVAE and other classifiers for case 1

class	Normal	Fault 4	Fault 13	AVG
TVAE	100	99.6	98.7	**99.43**
S1-DCNN	100	97.89	94.42	97.39
1-DCNN	100	92.11	95.94	96.01
SDAE	100	96.32	91.97	95.83
DBN	100	88.95	96.95	95.31

Table 7 Diagnosis results of TVAE and other classifiers for case 2

class	Normal	Fault 6	Fault 2	Fault 13	AVG
TVAE	100	96.23	100	98.7	**98.73**
S1-DCNN	100	95.51	93.37	95.73	96.06
1-DCNN	100	94.94	81.63	90.52	91.54
SDAE	100	98.32	74.49	92.89	90.1
DBN	100	84.271	93.37	81.52	89.58

unknown fault. As shown in Fig. 14, when the untrained fault 21 is input into the trained model, it is found that its probability distribution is relatively scattered, and its variance is mostly lower than 40. It can effectively isolate the unknown faults.

Table 8 Diagnosis results of TVAE and other classifiers for case 3

class	Normal	Fault 6	Fault 2	Fault 13	AVG
TVAE	100	94.17	98.16	96.13	**95.79**
S1-DCNN	100	81.44	79.90	80.56	85.55
1-DCNN	100	87.32	92.47	61.29	85.42
SDAE	100	84.88	82.80	60.75	82.16
DBN	100	80.49	92.47	55.91	82.29

In order to further verify the effectiveness of the algorithm in identifying unknown faults. There are twenty-one fault types of TE process for experiments in Table 9, we select one of the faults as test data, the remaining twenty faults as training data. First, the probability output and variance corresponding to 20 training faults are obtained by neuron-grouped convolution. Then when the output result of test fault is lower than the probability and variance of the training, it is defined as an unknown fault. Finally, the number of unknown faults detected/total number of test

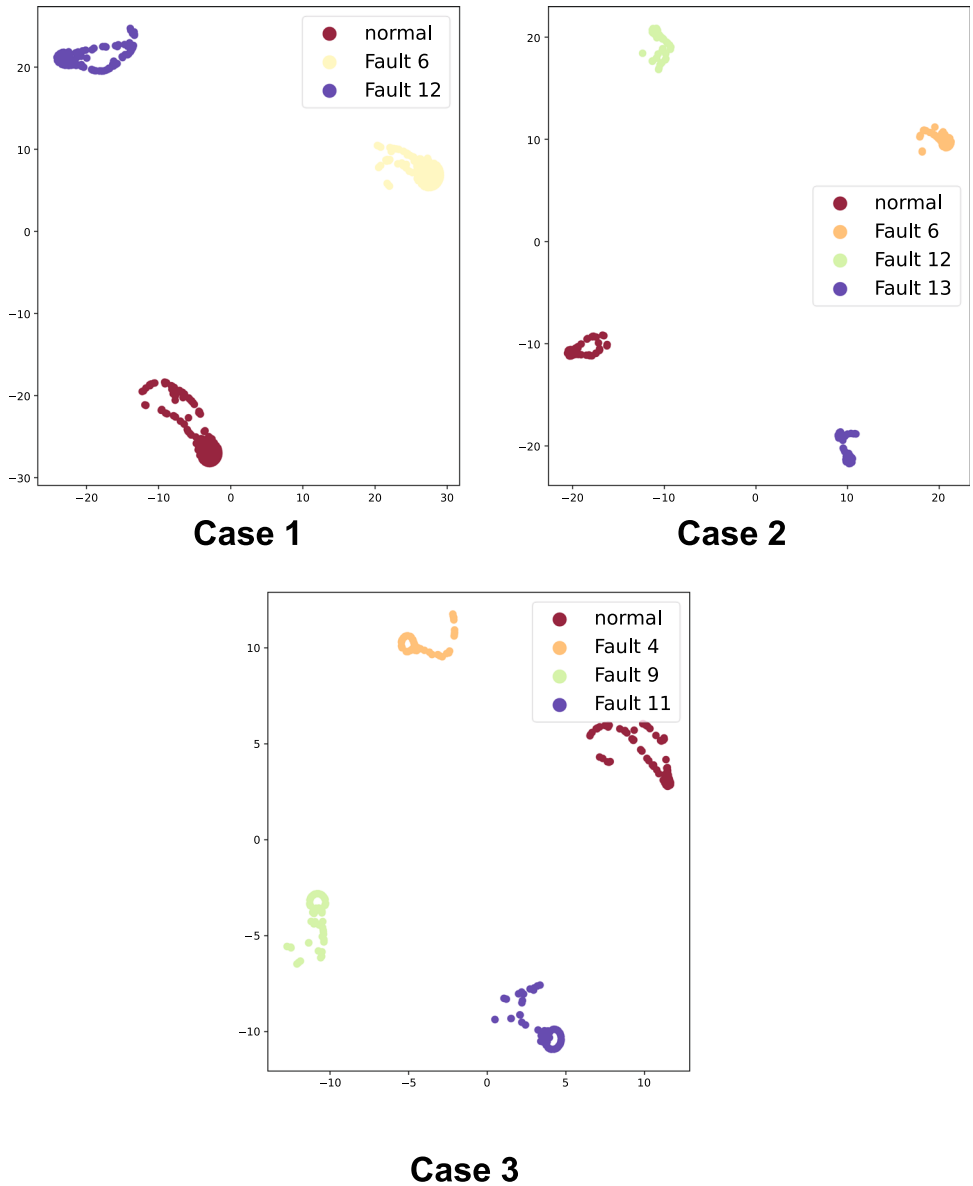

Fig. 13 Visualization of TVAE diagnostic results in three cases

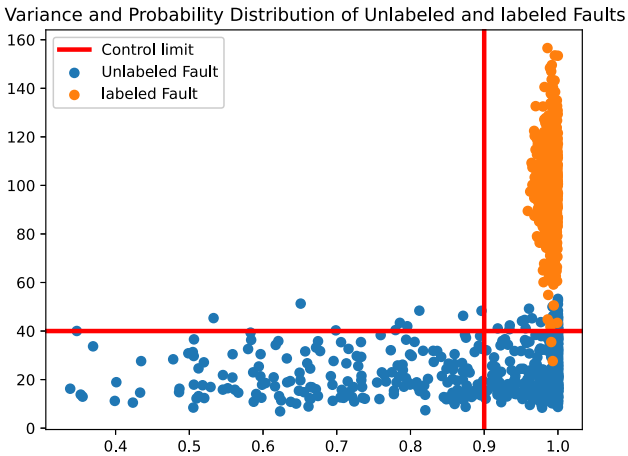

Fig. 14 The distribution of marked faults and unmarked faults

samples are recorded in Table 9 as the detection rate of unknown faults, and we compare the results with the results of algorithm [28, 29]. The experimental results are shown in Table 9. It can be seen that the performance of this algorithm is also better than the current research on identification of unknown faults.

The algorithm proposed in this paper is based on the correlation between variables and the time series of the system, so it is suitable for dynamic industry model, but not for static systems because the detection cycle is longer than the statistical analysis algorithm.

4.1 Ablation experiment

In the phase of fault detection, to verify the contribution of each part of the algorithm, we divide the algorithm into

Table 9 Comparison between TVAE and other algorithms in identification of unknown faults

Fault type	DSAE	DSSAE	CRAE	TVAE
Fault 1	98.5	99.2	**100**	98.7
Fault 2	98.2	98.2	97,4	**98.4**
Fault 4	93.7	99	**100**	**100**
Fault 5	99.2	99.5	**99.57**	97.1
Fault 6	99	99.7	**100**	99.3
Fault 7	99.7	99.7	**100**	**100**
Fault 8	97.2	95.5	84.3	**97.3**
Fault 10	59.2	72.5	83.30	**96.5**
Fault 11	83.2	84.5	**96.96**	85.5
Fault 12	59.2	**98.2**	90.89	95.9
Fault 13	84.5	94.5	88.07	**97.1**
Fault 14	94.7	99.25	**100**	99.2
Fault 16	62.7	74.7	78.96	**93.3**
Fault 17	97.5	**97.7**	95.44	90.1
Fault 18	90.2	90.5	80.48	**98.3**
Fault 19	73	78.5	**99.78**	92.8
Fault 20	85.1	87.2	90.89	**92.2**
Average	88.9	90.2	92.25	**95.09**

four states: only autoencoder, horizontally moving LSTM-AE, vertically moving LSTM-AE and TVAE, and verify the fault detection rate in each state. The results are shown in Table 10. From Table 10, we can see the effectiveness of the algorithm combination.

In order to further understand the role of TVAE model in feature learning at each stage, we introduce t-Stochastic Neighbor Embedding (t-SNE) [34] method for visual analysis of each step. We randomly selected 400 features that passed each step of training and mapped the extracted features into two-dimensional space, and draw fault scatter diagrams of different colors according to their labels. The visualization results are depicted in Fig. 15. Figure 15a shows the visualization results of unprocessed original features, and Fig. 15b shows the results after LSTM-autoencoder algorithm. Figure 15c shows the results of CNN without LSTM-autoencoder. The last Fig. 15d is the result after the TVAE that proposed in this paper. It can be clearly seen that the original data is disordered, and the information between various faults is difficult to identify. After the time information is extracted by LSTM-autoencoder, it can be seen that five types of faults can be successfully separated. When the LSTM-autoencoder layer is skipped and only CNN performed on the original data. It can be seen that most faults tend to be separated, but the distance between fault classes is too small, which is easy to affect the fault prediction results. As shown in Fig. 15d, when the fault feature is extracted by TVAE, the different

Table 10 Results of ablation experiment for TVAE fault detection

Fault type	AE	Horizontally moving LSTM-AE	Vertically moving LSTM-AE	TVAE
Fault 1	88.5	89.2	100	100
Fault 2	96.2	98.2	98,4	98.4
Fault 3	23.7	29	32.45	33.17
Fault 4	96.2	98.5	99.57	99.1
Fault 5	36.74	46.52	62.74	67.43
Fault 6	99.7	99.7	100	100
Fault 7	99.2	95.5	100	100
Fault 8	89.2	92.5	93.30	96.5
Fault 9	43.2	44.5	46.96	55.5
Fault 10	69.2	68.2	78.89	75.9
Fault 11	76.13	84.5	88.07	87.1
Fault 12	94.7	99.25	100	99.2
Fault 13	82.7	94.7	98.96	93.3
Fault 14	97.5	97.7	95.44	90.1
Fault 15	40.2	40.5	43.48	48.3
Fault 16	73	78.5	89.78	92.8
Fault 17	93.1	93.15	90.89	92.2
Fault 18	88.9	92.13	92.25	95.09
Fault 19	22.13	33.75	35.76	39.83
Fault 20	7.25	14.25	26.37	28.35
Fault 21	48.5	52.44	56.89	59.35

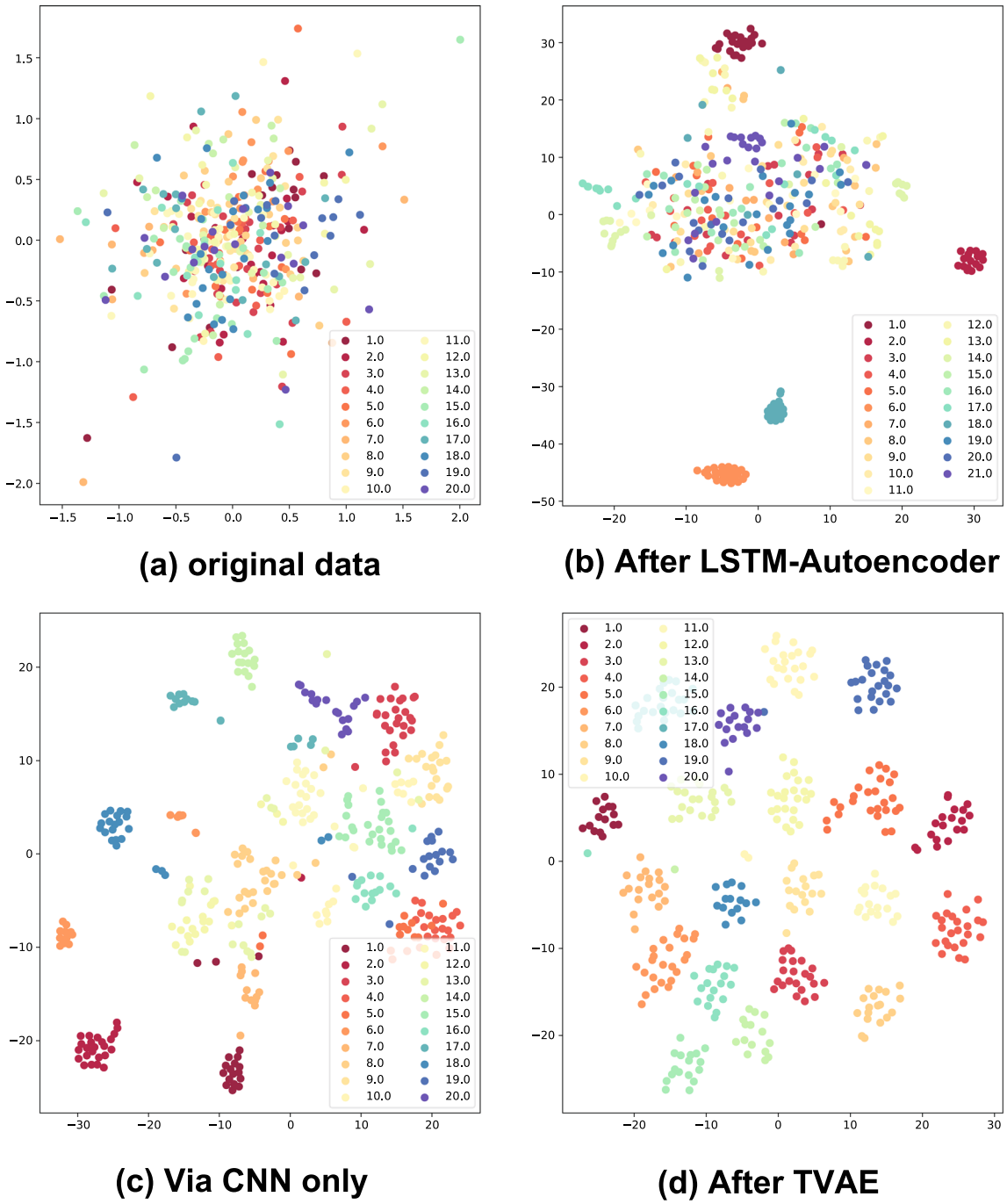

Fig. 15 t-SNE visual output of TVAE

faults can be clearly separated, which also proves that the combination of the algorithm is effective.

5 Conclusion

In this paper, a novel neuron-grouped two-view variational LSTM auto-encode model is proposed, which uses the two-view embedding to compressed time series information and captures the correlation between time variables, overcoming the problem that the current deep framework can not consider the relationship between different variables and time series at the same time. Through the pre-training of convolution groups and actual label, the distance between different fault types is further expanded and the classification results are clearer. In addition, it classifies faults based on hierarchical fault correlation feature extraction, and can judge unknown faults by the probability and

variance of the output, which is a new fault detection and diagnosis paradigm. The simulation results of TE process show that the proposed method has excellent performance in diagnosis of complex and similar fault types.

As the depth of the neural network increases, computational pressure increases and equipment requirements become higher. This may cause the algorithm to have time delay, resulting in the old fault not being trained and new fault being generated. Therefore, it is urgent to develop lightweight network architecture.

Acknowledgements This work was partially supported by the National Science Foundation of China (62271293, 61903238), the Natural Science Foundation of Shandong Province, PR China (ZR2021MF035), the Social Science Planning Project of Shandong Province, PR China (22CYYJ13).

Data availability The datasets analysed during the current study are available in the Tennessee Eastman Challenge Archive, http://depts.washington.edu/control/LARRY/TE/download.html.

Declarations

Conflict of interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

1. Abid A, Khan M, Iqbal J (2021) A review on fault detection and diagnosis techniques: basics and beyond. Artif Intell Rev 54:1–26. https://doi.org/10.1007/s10462-020-09934-2
2. Wachowicz A, Pytlik J, Małysiak-Mrozek B, Tokarz K, Mrozek D (2022) Edge computing in iot-enabled honeybee monitoring for the detection of. Int J Appl Math Comput Sci 32(3):355–369. https://doi.org/10.34768/amcs-2022-0026
3. Liu B, Huang P, Zeng X, Li Z (2017) Hidden defect recognition based on the improved ensemble empirical decomposition method and pulsed eddy current testing. NDT & E Int 86:175–185. https://doi.org/10.1016/j.ndteint.2016.12.009
4. Sun R-B, Yang Z-B, Gryllias K, Chen X-F (2020) Cyclostationary modeling for local fault diagnosis of planetary gear vibration signals. J Sound Vib 471:115175. https://doi.org/10.1016/j.jsv.2020.115175
5. Qin SJ (2012) Survey on data-driven industrial process monitoring and diagnosis. Annu Rev Control 36:220–234. https://doi.org/10.1016/j.arcontrol.2012.09.004
6. Wang T, Qiao M, Zhang M, Yang Y, Snoussi H (2020) Data-driven prognostic method based on self-supervised learning approaches for fault detection. J Intell Manuf 31:1611–1619. https://doi.org/10.1007/s10845-018-1431-x
7. Wang J, Gui G, Wang R, Yin Y, Huang H, Wang Y (2020) Deep learning based exploring channel reciprocity method in fdd systems. In: Liang Q, Wang W, Liu X, Na Z, Jia M, Zhang B (eds) Communications, signal processing, and systems. Springer, Singapore, pp 2533–2541
8. Moustapha A, Selmic R (2017) Wireless sensor network modeling using modified recurrent neural networks: application to fault detection. IEEE Trans Instrum Meas 57:313–318. https://doi.org/10.1109/TIM.2007.913803
9. Yuan H, Chen G (2016) Fault diagnosis in nonlinear circuit based on Volterra series and recurrent neural network. Int Conf Neural Inf Process 4234:518–525. https://doi.org/10.1007/11893295_57
10. Zia T, Zahid U (2019) Long short-term memory recurrent neural network architectures for urdu acoustic modeling. Int J Speech Technol 22:21–30. https://doi.org/10.1007/s10772-018-09573-7
11. Yang R, Huang M, Lu Q, Zhong M (2018) Rotating machinery fault diagnosis using long-short-term memory recurrent neural network. IFAC Pap OnLine 51:228–232. https://doi.org/10.1016/j.ifacol.2018.09.582
12. Yuan M, Wu Y, Lin L (2016) Fault diagnosis and remaining useful life estimation of aero engine using lstm neural network. In: 2016 IEEE international conference on aircraft utility systems (AUS), vol 10, pp 135–140. https://doi.org/10.1109/AUS.2016.7748035
13. Lui N, Wang X (2008) Fault diagnosis based on signed digraph combined with dynamic kernel pls and svr. Ind Eng Chem Res IND ENG CHEM RES 47:3543–3562. https://doi.org/10.1021/ie8009457
14. Choi S, Lee C, Lee J-M, Park J, Lee I-B (2015) Fault detection and identification of nonlinear processes based on kpca. Chemom Intell Lab Syst 75:55–67. https://doi.org/10.1016/j.chemolab.2004.05.001
15. Cheng H, Liu Y, Huang D, Cai B, Wang Q (2021) Rebooting kernel cca method for nonlinear quality-relevant fault detection in process industries. Process Saf Environ Prot 149:619–630. https://doi.org/10.1016/j.psep.2021.03.025
16. Vincent P, Larochelle H, Bengio Y, Manzagol P-A (2008) Extracting and composing robust features with denoising autoencoders. In: Proceedings of the 25th international conference on machine learning vol 01, pp 1096–1103. https://doi.org/10.1145/1390156.1390294
17. Chen Z, Li W (2017) Multisensor feature fusion for bearing fault diagnosis using sparse autoencoder and deep belief network. IEEE Trans Instrum Meas 03:1–10. https://doi.org/10.1109/TIM.2017.2669947
18. Vincent P, Larochelle H, Bengio Y, Manzagol P-A (2008) Extracting and composing robust features with denoising autoencoders. In: Proceedings of the 25th international conference on machine learning, pp 1096–1103. https://doi.org/10.1145/1390156.1390294
19. Xu F, tai Peter Tse W, Tse YL (2018) Roller bearing fault diagnosis using stacked denoising autoencoder in deep learning and gath-geva clustering algorithm without principal component analysis and data label. Appl Soft Comput 73:898–913. https://doi.org/10.1016/j.asoc.2018.09.037
20. Lee SM, Park S-Y, Choi B-H (2022) Application of domain-adaptive convolutional variational autoencoder for stress-state prediction. Knowl Based Syst. https://doi.org/10.1016/j.knosys.2022.108827
21. Bedja-Johnson Z, Wu P, Grande D, Anderlini E (2022) Smart anomaly detection for slocum underwater gliders with a variational autoencoder with long short-term memory networks. Appl Ocean Res 120:103030. https://doi.org/10.1016/j.apor.2021.103030
22. Huang L, Ren H, Chai Y, Qu J (2021) A fault detection method based on stacking the sae-srbm for nonstationary and stationary hybrid processes. Int J Appl Math Comput Sci 31(1):29–43. https://doi.org/10.34768/amcs-2021-0003
23. Yu M, Quan T, Peng Q, Yu X, Liu L (2022) A model-based collaborate filtering algorithm based on stacked autoencoder. Neural Comput Appl 513:2503–2511. https://doi.org/10.1007/s00521-021-05933-8
24. Lu C, Wang Z-Y, Qin W-L, Ma J (2017) Fault diagnosis of rotary machinery components using a stacked denoising autoencoder-

based health state identification. Signal Process 130:377–388. https://doi.org/10.1016/j.sigpro.2016.07.028
25. Li Z, Zhao Y, Han J, Su Y, Jiao R, Wen X, Pei D (2021) Multivariate time series anomaly detection and interpretation using hierarchical inter-metric and temporal embedding. In: Proceedings of the 27th ACM SIGKDD conference on knowledge discovery and data mining. KDD'21, pp. 3220–3230. Association for Computing Machinery, New York, NY, USA. https://doi.org/10.1145/3447548.3467075
26. Liu X, Yu J, Ye L (2021) Residual attention convolutional autoencoder for feature learning and fault detection in nonlinear industrial processes. Neural Comput Appl 33:12737–12753. https://doi.org/10.1007/s00521-021-05919-6
27. Zhijiang L, Wang Y (2021) New nonlinear approach for process monitoring: neural component analysis. Ind Eng Chem Res 60:387–398. https://doi.org/10.1021/acs.iecr.0c02256
28. Jiang L, Ge Z, Song Z (2017) Semi-supervised fault classification based on dynamic sparse stacked auto-encoders model. Chemom Intell Lab Syst 168:72–83. https://doi.org/10.1016/j.chemolab.2017.06.010
29. Ma Y, Shi H, Tan S, Tao Y, Song B (2022) Consistency regularization auto-encoder network for semi-supervised process fault diagnosis. IEEE Trans Instrum Meas 71:1–15. https://doi.org/10.1109/TIM.2022.3184346
30. Yin S, Ding SX, Haghani A, Hao H, Zhang P (2012) A comparison study of basic data-driven fault diagnosis and process monitoring methods on the benchmark tennessee eastman process. J Process Control 22:1567–1581. https://doi.org/10.1016/j.jprocont.2012.06.009
31. Pan Z, Wang Y, Yuan X, Yang C, Gui W (2021) A classification-driven neuron-grouped sae for feature representation and its application to fault classification in chemical processes. Knowl Based Syst 230:107350. https://doi.org/10.1016/j.knosys.2021.107350
32. Deng L, Zhang Y, Dai Y, Ji X, Zhou L, Dang Y (2021) Integrating feature optimization using a dynamic convolutional neural network for chemical process supervised fault classification. Process Saf Environ Prot 155:473–485. https://doi.org/10.1016/j.psep.2021.09.032
33. Yu Jianbo ZC, Shijin W (2022) Sparse one-dimensional convolutional neural network-based feature learning for fault detection and diagnosis in multivariable manufacturing processes. Neural Comput Appl 34:4343–4366. https://doi.org/10.1007/s00521-021-06575-6
34. Van der Maaten L, Hinton G (2008) Visualizing data using t-sne. J Mach Learn Res 9:2579–2605

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.